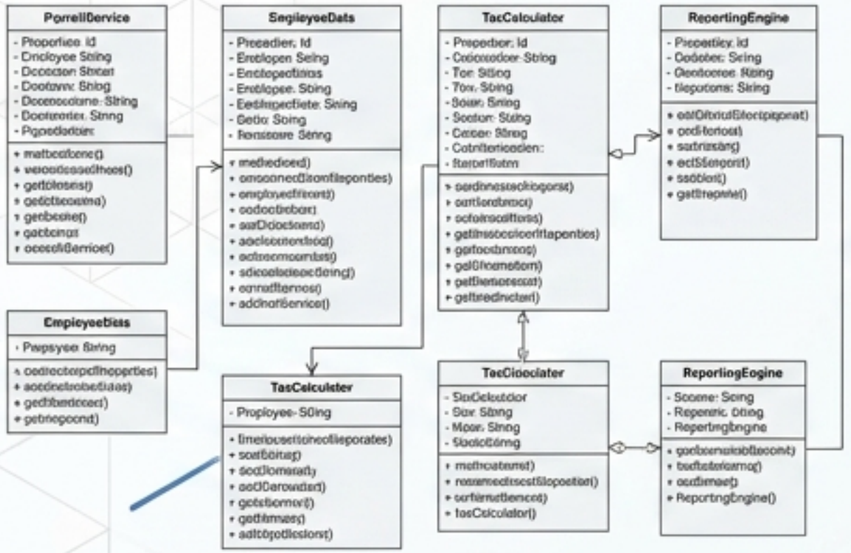
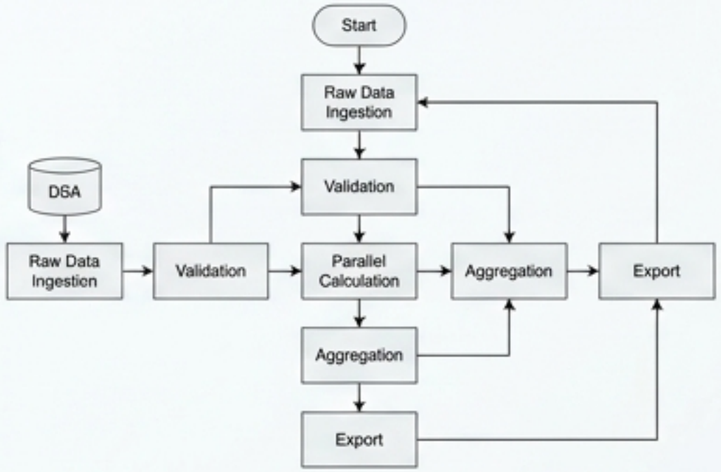
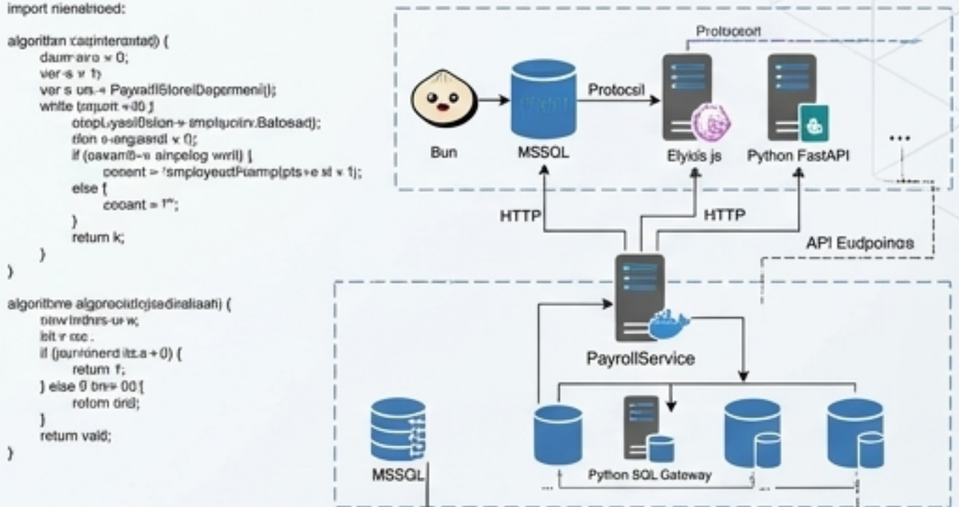
PT Rebinmas

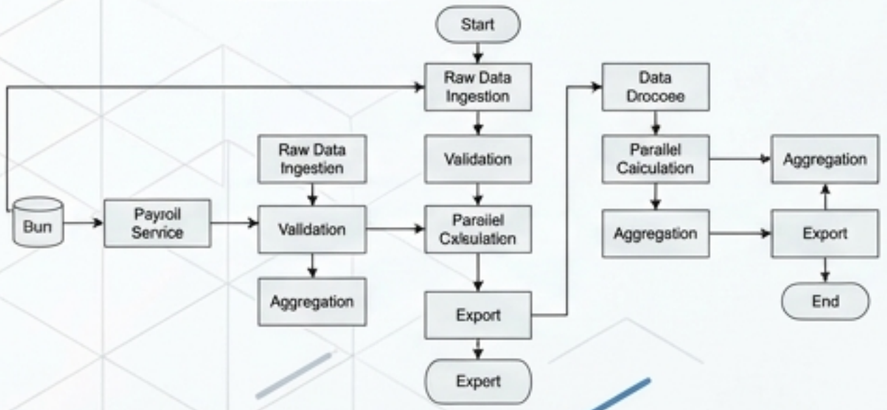
Payroll Daftar Upah Reporting System

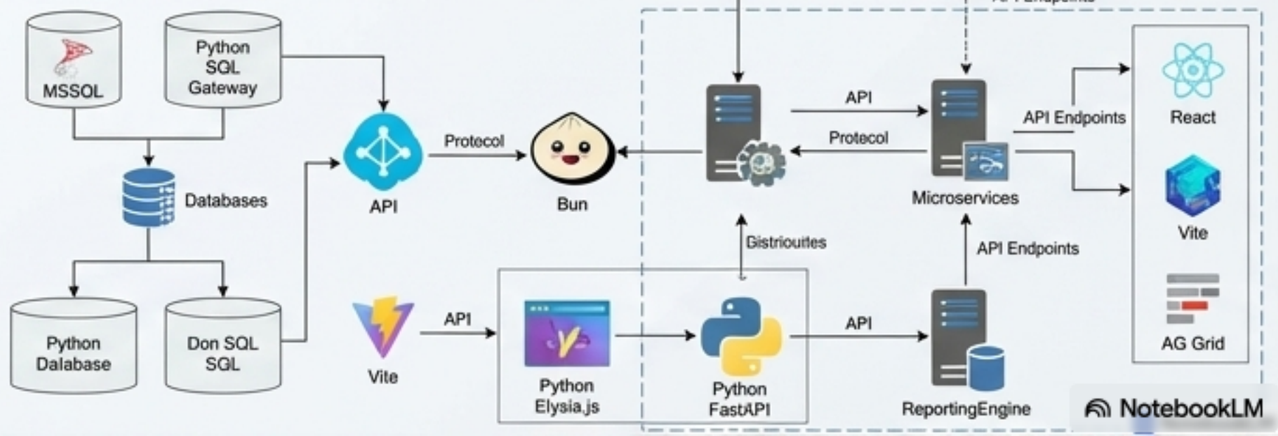
Technical Blueprint & Architecture V1.0 - 2026











NotebookLM

System Scale and Operational Impact

Delivering accurate, real-time, and highly exportable payroll reporting for PT Rebinmas through a secure, multi-tier architecture.

25+

Backend Services

Powered by `Bun + Elysia.js`

50+

API Endpoints

Handling `parallel` data `extraction`

15+

Frontend Pages

`React + Vite with AG Grid Enterprise`

20+

Database Tables

Managed via `Python FastAPI SQL Gateway`

NotebookLM

The Multi-Service Mental Model

Demystifying the architecture through the 'Restaurant' analogy.

The Menu & Cashier (Frontend)

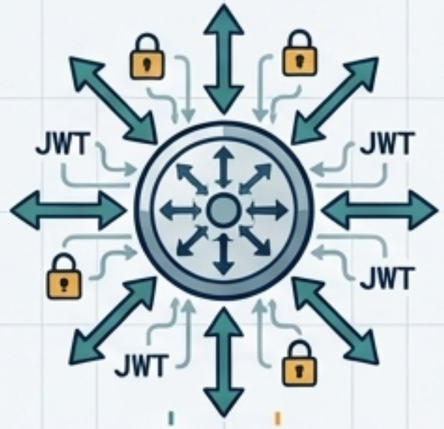
`React + Vite + AG Grid`

Where users interact, filter, and view the final data.

The Manager (Backend)

`Bun + Elysia.js`

Orchestrates the flow, handles JWT authentication, and directs traffic.



The Warehouse (Database)

`MSSQL + SQL Gateway`

Securely stores raw HR data, attendance, and transactions without direct backend exposure.

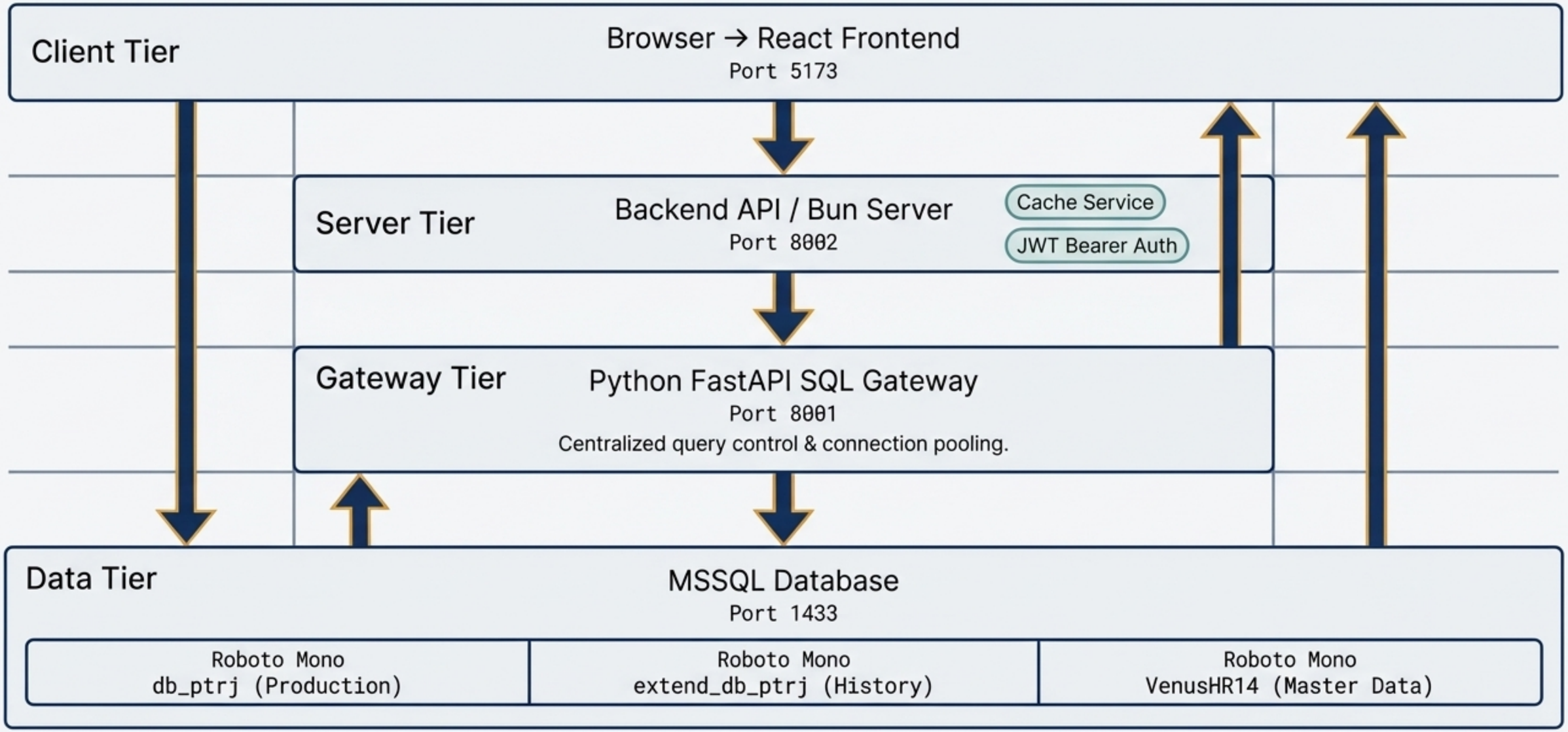
The Specialists (Microservices)

`Python FastAPI & Calculation Engines`

Handle specific heavy-lifting like PPh21 tax formulas and data aggregation.

NotebookLM

3-Tier Technical Architecture and Routing



NotebookLM

Core Engine I: Tier-Based Overtime (Lembur)

The UPJ Formula

$$\text{UPJ (Upah Per Jam)} = \frac{\text{pay\_rate} \times 30}{173}$$

Default UPJ fallback: 17,257

Day Type	Tier 1	Tier 2	Tier 3	Boundary
Workday (Long/Short)	1.5x	2x	-	1 hour
Sunday	2x	3x	4x	5/7 hours
Holiday (Regular)	2x	3x	4x	5/7 hours
Holiday (Religious)	3x	4x	4x	5/7 hours

NotebookLM

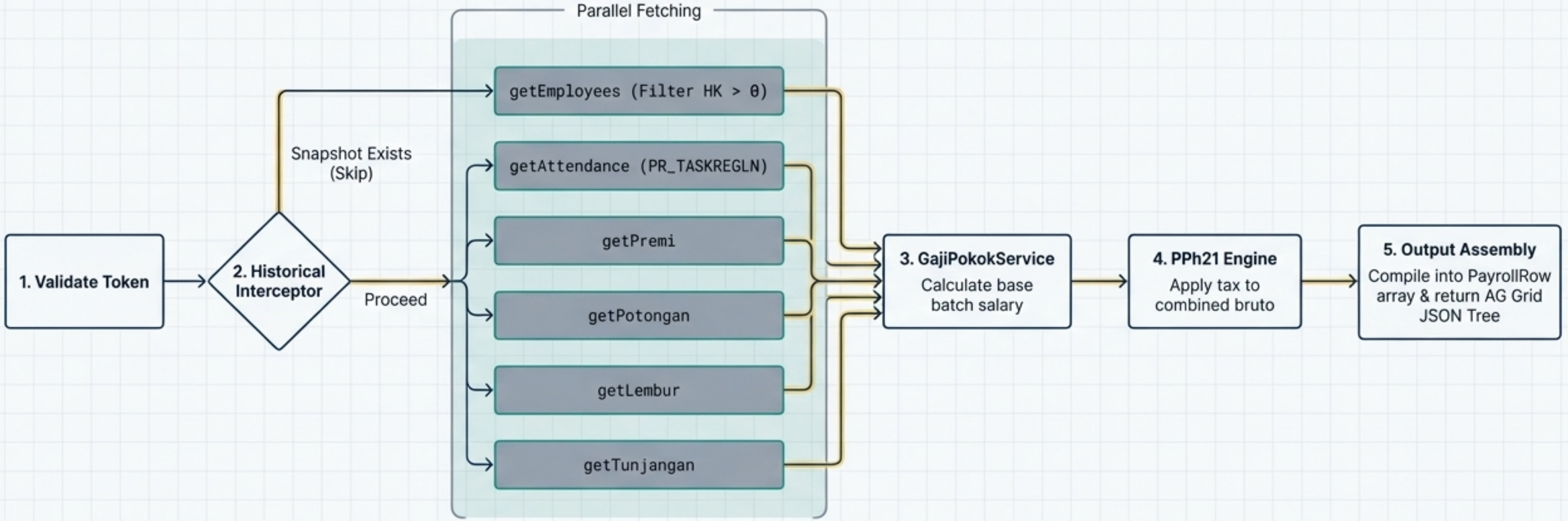
Core Engine II: PPh21 Tax via the TER Method

The Rice Ration Mapping Table

Rice Ration	PTKP Status	TER Category
2250	TK/0	TER A (5%)
3250	TK/1	TER A (5%)
3750	K/0	TER A (5%)
4200	TK/2	TER B (15%)
4650	K/1	TER B (15%)
5550	K/2	TER B (15%)
6450	K/3	TER C (25%)

NotebookLM

The Data Extraction Pipeline



NotebookLM

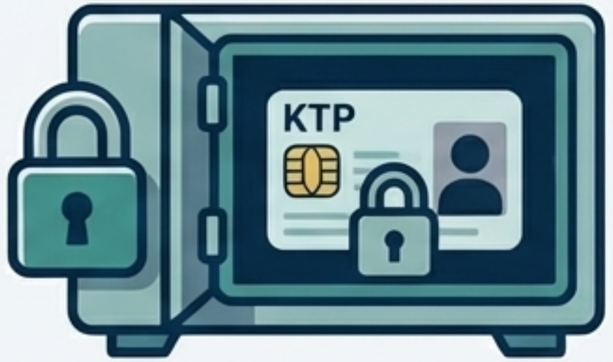
Historical Data Preservation Architecture

The Problem

`EmpCode: A012 → A099`

`EmpCode` mutates when an employee changes divisions. Re-running live calculations on old months breaks the historical accuracy of reports.

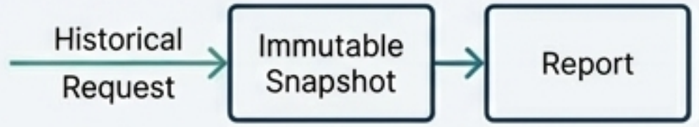
The Snapshot Architecture



The system triggers a Snapshot upon month-closing. Data is saved to `payroll_history_header` and `detail` in `extend_db_ptrj`.

Permanent Tracking

The system intercepts requests for past months and pulls the exact, immutable snapshot locked to the employee's permanent NIK (KTP), completely bypassing live recalculation.

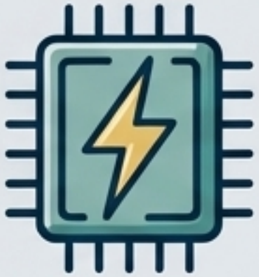


NotebookLM

Performance and Optimization Strategy

Parallel Processing (Threading)

Utilizes `ThreadedDataExtractor` and `ThreadedHeaderService` to extract payroll data and generate dynamic AG grid headers simultaneously, slashing response times.



Intelligent Caching

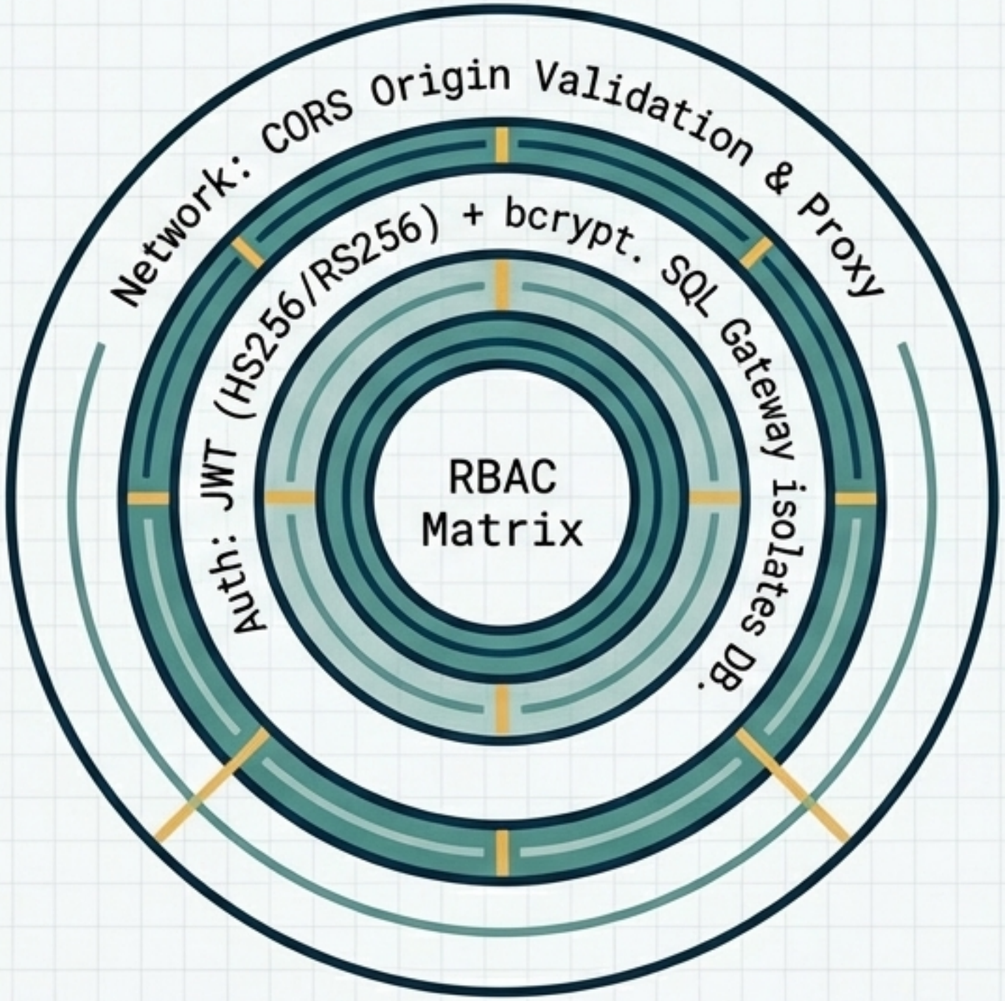
Singleton `CacheService` with a configurable 120-second TTL. Caches heavy `/payroll/report` endpoints to prevent redundant database strain.

Database Batching

Utilizes the `SQL Gateway` for connection pooling, batch queries, and parameterized statements (? placeholders) to optimize memory and prevent SQL injection.

NotebookLM

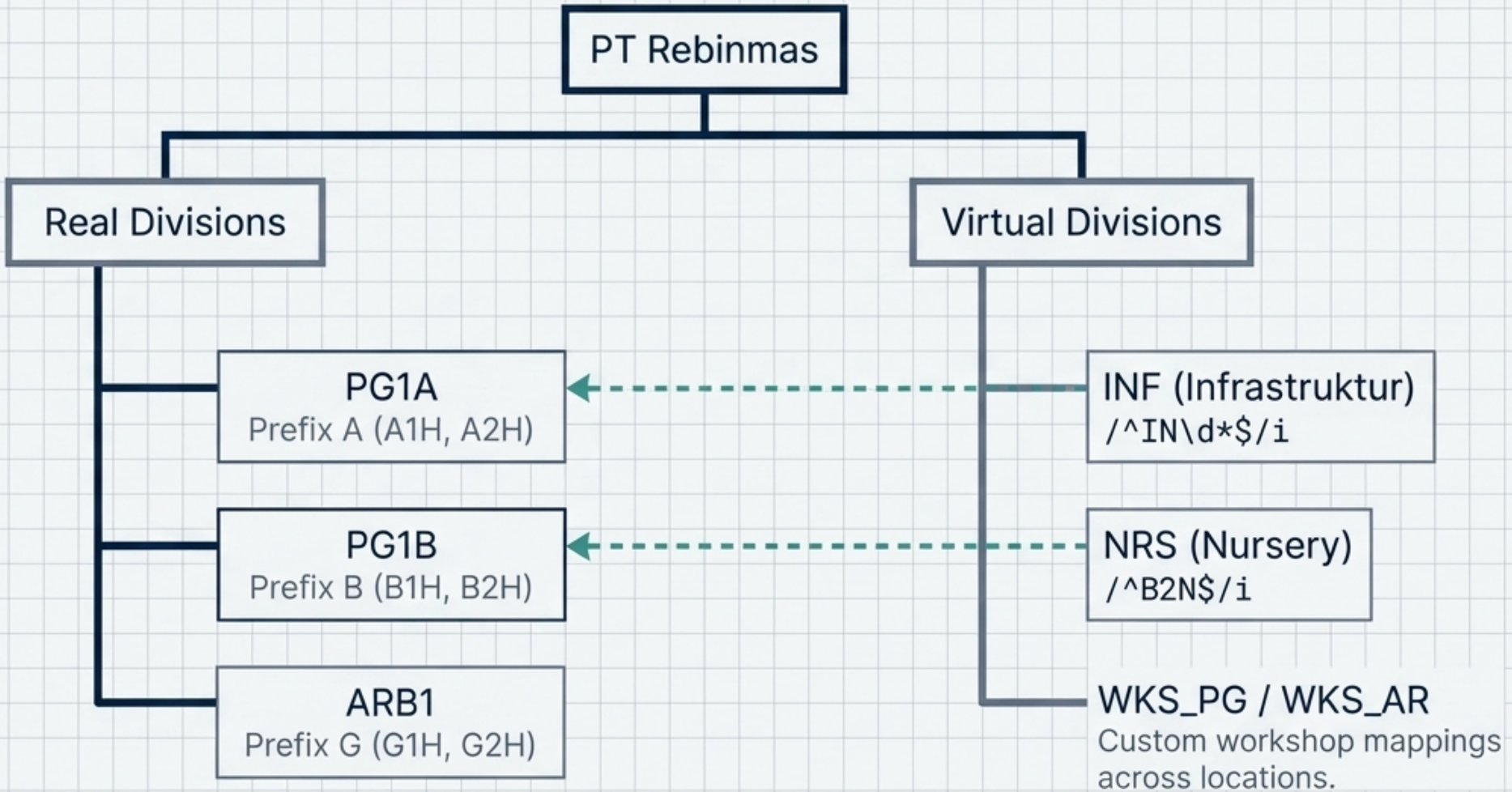
Security and Access Control Layers



Role	Access Level
ADMIN	Full access to all divisions and features.
MANAGER	Access restricted to specifically assigned divisions/gangs.
VIEWER	Global read-only access.

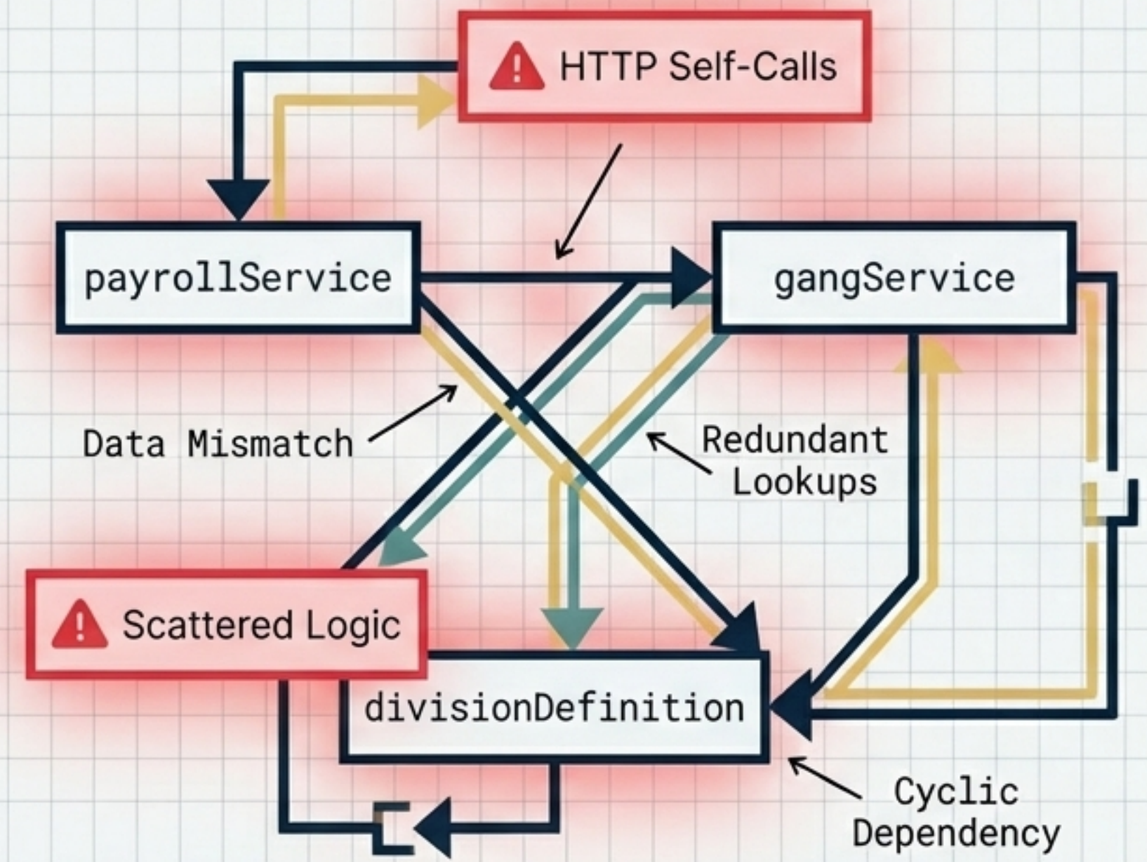
NotebookLM

Division Configuration and Gang Mapping



NotebookLM

The Case for Refactoring: Current Challenges



Data Integrity Risks

Multiple sources of truth exist for division mappings, risking incorrect data rendering in reports.

Calculation Inconsistencies

Payroll formulas (like BPJS and specific allowances) are scattered across more than 5 different services.

Performance Overhead

Mixed data access patterns and HTTP self-calls inside the backend cause unnecessary latency and complex debugging.

NotebookLM

Target Architecture: Centralization

DivisionConfigService
(The Map)
A strict Single Source of Truth for all real/virtual divisions, aliases, and gang pattern matching.

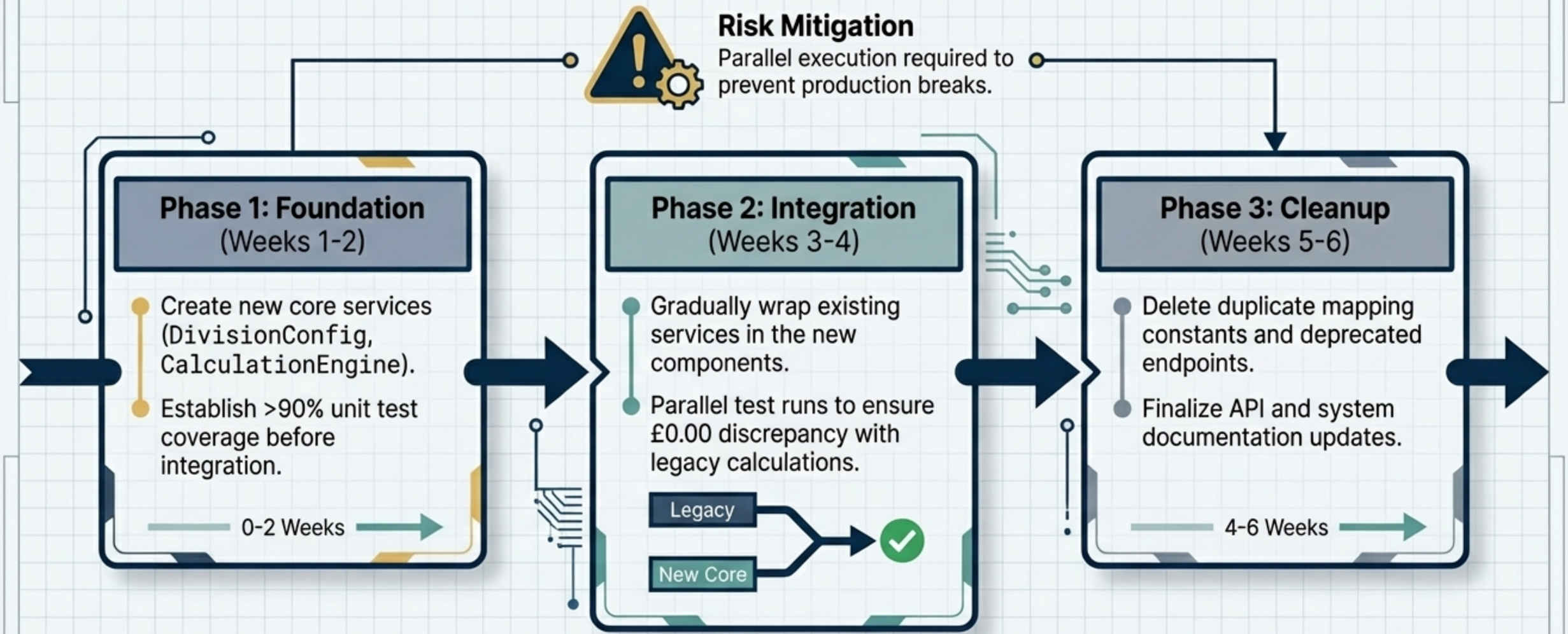
UNIFIED SYSTEM CORE

PayrollCalculationEngine
(The Brain)
One centralized location for all salary, tax (PPh21), and BPJS formulas ensuring 100% calculation consistency.

PayrollRepository
(The Vault)
A unified data access layer that entirely eliminates internal HTTP self-calls, managing all cache and database transactions directly.

NotebookLM

Implementation and Migration Strategy



NotebookLM

2026 Roadmap and Future Capabilities

You are here

Q1 2026 (Completed)

- PPh21 TER implementation
- Google Spreadsheet Sync
- Core calculation fixes

Q2 2026 (Upcoming)

- Mobile Responsive UI Design
- Real-time Websocket Notifications
- Automated PDF Report Generation

Q3 2026 (Upcoming)

- Service Worker Offline Mode
- External HRIS System Integrations
- Machine Learning Payroll Predictions

NotebookLM